Operand addressing - brief recap

- We’ve seen two forms of addressing operands:
  - Base-displacement (base address plus 12-bit displacement)
  - Long displacement (base address plus 20-bit displacement)
  - We will now see a third form, called **relative addressing**

Relative addressing

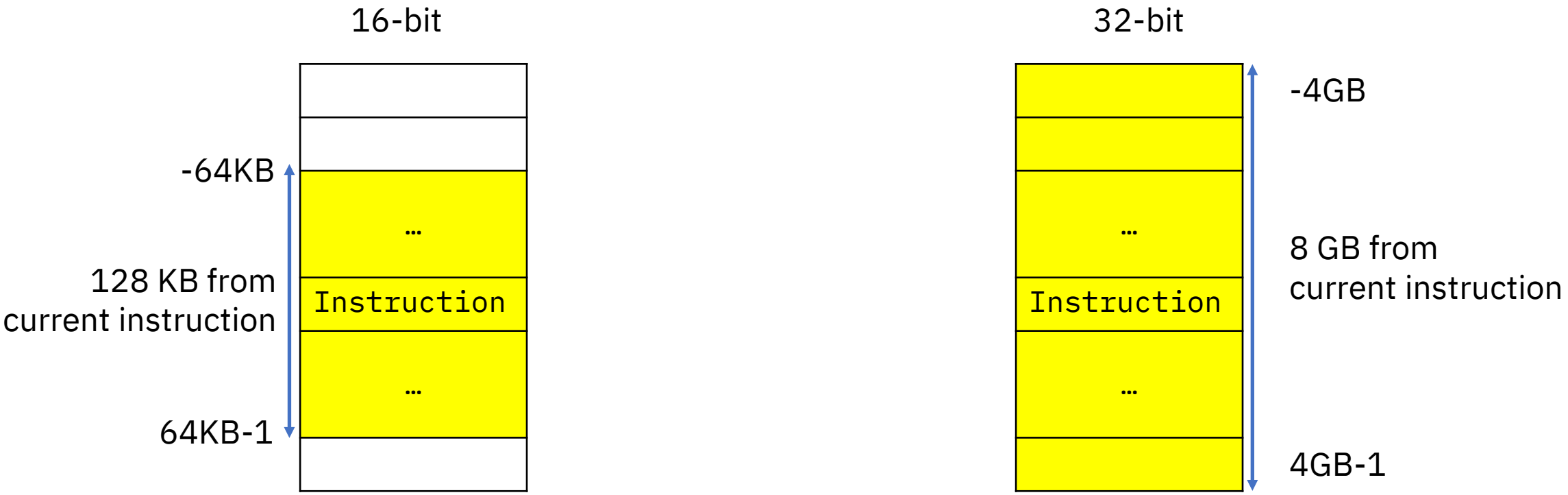
- Unlike base-displacement and long displacement, *relative addressing does not use a base register*

- Instructions that use relative addressing have 16-bit or 32-bit operands that represent a *signed displacement from the address of the instruction being executed*

- The signed displacements represent the number of *halfwords* before or after the instruction
  - The effective address is always even (that is, on a halfword boundary), because instructions are always halfword-aligned

Relative addressing

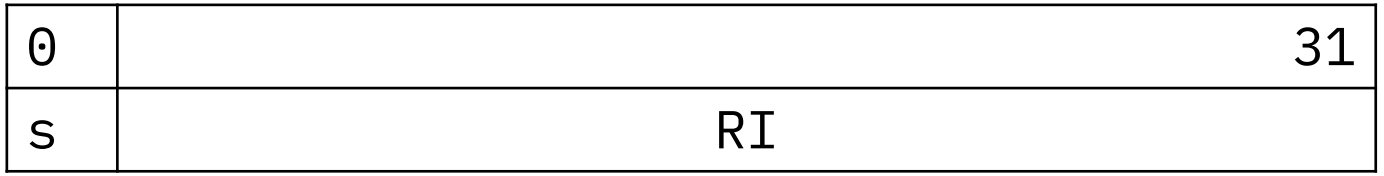
- Comparing 16-bit and 32-bit relative addressing:



Relative addressing - operand format

- Relative address displacements look like this in an instruction:
- 16-bit:

- 32-bit:



- Note “RI”: *relative-immediate.* Operands are called *immediate* when they are the actual values, instead of a register or memory reference.

Relative addressing

- Same as with long displacement, there are versions of instructions that use it.
- The instruction that you will most probably use most frequently is **Jump**, a relative addressing version of Branch.
- The 16-bit version is called *Jump*
- The 32-bit version is called *Jump Long*
- Next, an example

Relative addressing - Example

- Our example program, now using relative addressing:

Address	Contents
2A000	...
2A006	**Jump** to Calc_Tax
…	
…	
…	
2C080	Calc_Tax: …
…	

Opcode	Operand (Calc_Tax)
Jump	103D

Why 103D?

- 2C080 - 2A006 = 207A bytes
- 207A / 2 = 103D *halfwords*

Note: as a programmer you don't need to do these calculations; the Assembler does. This "just works"!

Relative addressing - Summary

- Relative addressing doesn’t use base registers
- It uses 16-bit or 32-bit displacements
  - 16-bit displacements have a 128 KB range
  - 32-bit displacements have an 8GB range
- Displacement units are halfwords (2 bytes on an even address)
- Displacements are relative to the instructions being executed
- Displacements are signed (half the addressing range is before the instruction and, the other half, beyond)

What next?

- We learned about operand addressing
- The instruction formats we saw were simplifications, not the actual formats
- Time to learn the actual instruction formats